Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Name	Joshua	Registration No.	24110085
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Batch	2
Subject Code & Name	CS3807 – Deep Learning Laboratory	Semester / AY	V / 2026–27

1. Objective

To implement and compare Vanilla RNN, LSTM and GRU for sequence classification, verify Back-propagation Through Time numerically, study the effect of sequence length, build a CNN–recurrent pipeline for video understanding and train a small encoder–decoder for sequence-to-sequence learning.

2. Dataset and Experimental Setup

Sequence Dataset

Six activity classes were used: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. The UCI HAR archive could not be downloaded inside the execution environment, so a HAR-style set of inertial sequences (120 windows per class) was generated in the notebook. Dynamic classes are periodic multi-channel signals and static classes are low-amplitude signals with different offsets. Each window has 128 time steps and 9 channels:

$$X \in \mathbb{R}^{N\times 128\times 9}, \qquad X_{\text{batch}} \in \mathbb{R}^{32\times 128\times 9}.$$

Dimension	Meaning	Value
N	Number of sequences	720
T	Time steps per sequence	128
F	Features/channels per time step	9

Preprocessing

1. Arranged every observation window as 128×9 and assigned one activity label.
2. Encoded the six labels as integers 0–5.
3. Created a stratified 70% / 15% / 15% train / validation / test split (seed 42).
4. Normalized every channel with the mean and standard deviation of the training set only.
5. Verified the class distribution. The test set was not used for model selection.

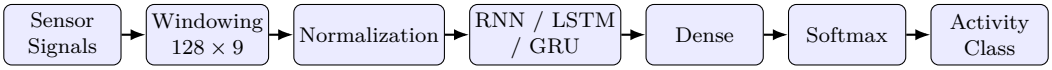
Preprocessing output

```
Input tensor shape:
Training   : (504, 128, 9)
Validation : (108, 128, 9)
Testing    : (108, 128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

Class distribution

Class	Train	Val	Test
WALKING	84	18	18
WALKING_UPSTAIRS	84	18	18
WALKING_DOWNSTAIRS	84	18	18
SITTING	84	18	18
STANDING	84	18	18
LAYING	84	18	18
Total	504	108	108

3. Overall Experimental Pipeline



All three recurrent models used the same input, split, output layer, optimizer, batch size and evaluation protocol.

4. Temporal Data Visualization

Plot 1: Sensor Signal versus Time

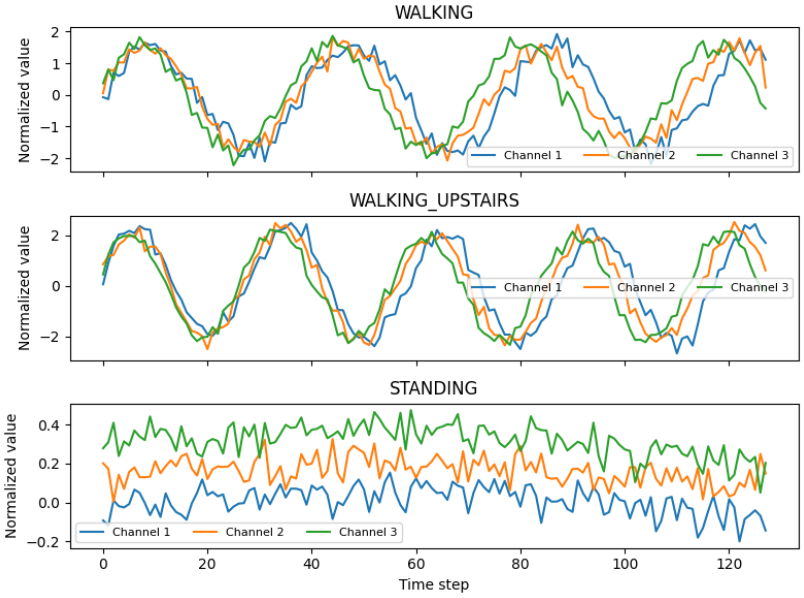


Figure 1: Three normalized channels for WALKING, WALKING_UPSTAIRS and STANDING.

Inference: (1) Walking classes show regular oscillations, while STANDING stays almost flat with small noise. (2) WALKING and WALKING_UPSTAIRS look very similar; the static classes also resemble each other. (3) The class is defined by how the signal changes over time, so shuffling the time steps would destroy the pattern.

5. Vanilla RNN and BPTT Numerical Exercise

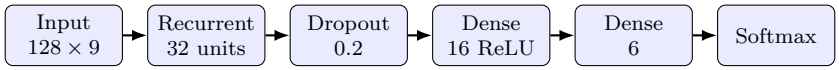
A Vanilla RNN updates $h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$ and the final hidden state is passed to the classifier. BPTT multiplies the derivative $W_h(1 - h_t^2)$ once per step, so gradients can vanish

($|\partial h_t/\partial h_{t-k}| \to 0$) or explode over long sequences.
With $x = (0.5, 0.7, 0.2)$, $h_0 = 0$, $W_x = 0.5$, $W_h = 0.8$, $b = 0.1$:

t	x_t	$W_x x_t + W_h h_{t-1} + b$	h_t (manual)	h_t (program)
1	0.5	$0.25 + 0 + 0.1 = 0.350000$	0.336376	0.336376
2	0.7	$0.35 + 0.269101 + 0.1 = 0.719101$	0.616352	0.616352
3	0.2	$0.10 + 0.493082 + 0.1 = 0.693082$	0.599958	0.599958

Inference: The manual and program values match. Each step reuses the same W_h, which is why the gradient is a product over time steps.

6. Model Architecture and Training



Parameter	Used	Manual
Recurrent units	32	32
Dropout	0.2	0.2
Optimizer	Adam	Adam
Learning rate	10^{-3}	10^{-3}
Batch size	32	32
Epochs	8	30
Loss	Cross-entropy	Sparse CE
Gradient clipping	norm 5.0	–

Model	Layer	Units	Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

LSTM: the forget gate f_t removes old memory, the input gate i_t writes the candidate $\tilde{C}_t$, and the output gate o_t exposes $h_t = o_t \tanh(C_t)$. Because $C_t = f_t C_{t-1} + i_t \tilde{C}_t$ is updated additively, information and gradients can pass across many steps.
GRU: the update gate z_t mixes old and new state, $h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t$, and the reset gate r_t controls how much history enters $\tilde{h}_t$. It has no separate cell state.

7. Training Plots

Plot 2 and Plot 3: Training and Validation Loss / Accuracy

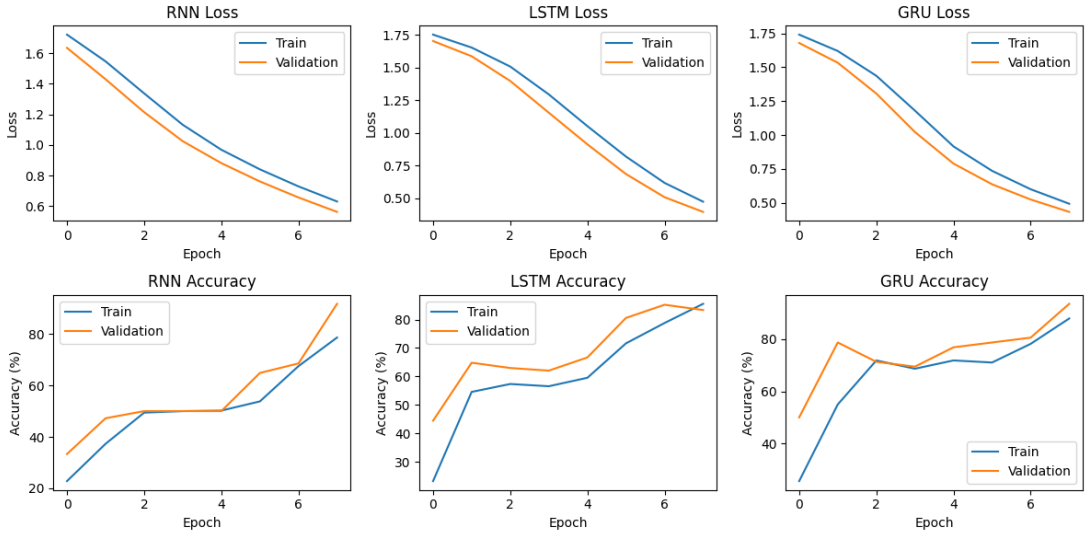


Figure 2: Loss (top) and accuracy (bottom) per epoch for RNN, LSTM and GRU.

Inference: All three losses were still falling at epoch 8, so none had fully converged. **RNN** improved slowly and stayed near 50% accuracy for several epochs. **LSTM** rose steadily to about 85%. **GRU** rose fastest and ended highest. Validation curves stayed at or above training curves, so there is no overfitting; the short training suggests mild underfitting.

8. Performance Evaluation

Metric	RNN	LSTM	GRU
Accuracy (%)	86.11	83.33	**89.81**
Macro Precision (%)	**92.42**	75.00	90.79
Macro Recall (%)	86.11	83.33	**89.81**
Macro F1 (%)	83.19	77.78	**89.55**
Parameters	2,006	6,134	4,758
Training Time (s)	1.060	1.169	2.561

9. Confusion Matrix Analysis

Plot 4: Confusion Matrix

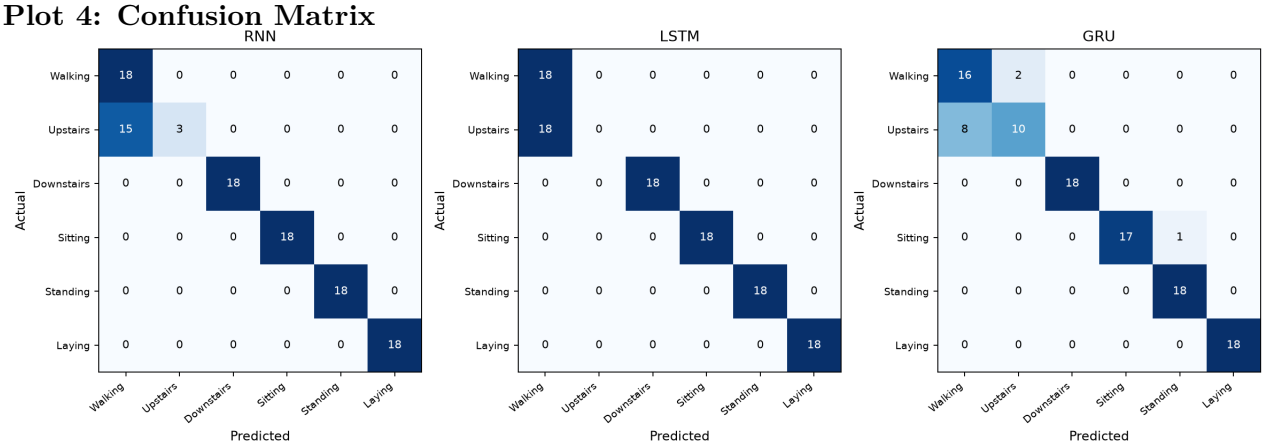


Figure 3: Test confusion matrices (18 test windows per class).

- **Highest recognition:** WALKING_DOWNSTAIRS, STANDING and LAYING were 18/18 for all three models.
- **Most misclassified:** WALKING_UPSTAIRS (RNN 15, LSTM 18, GRU 8 errors).
- **Confused pair:** WALKING ↔ WALKING_UPSTAIRS; GRU also confused one SITTING window with STANDING.
- **Consistency:** the same pair failed in all models; only the severity changed.

Inference: Accuracy alone hides that LSTM never predicted WALKING_UPSTAIRS, which is why its macro precision is only 75%.

10. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	2,006	6,134	4,758
Training time (s)	1.060	1.169	2.561
Test F1-score (%)	83.19	77.78	89.55

Plot 5: Model Performance Comparison

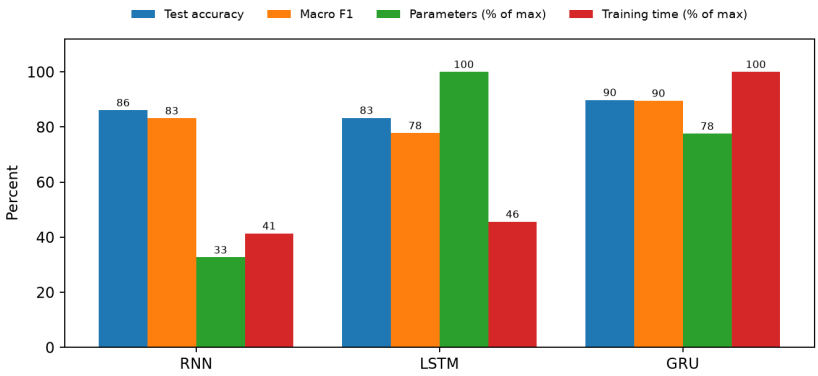


Figure 4: Test accuracy, macro F1, and parameters / training time normalized to the largest value.

Inference: LSTM has about 3× the RNN parameters (four gated transforms) and GRU about 2.4× (three). GRU gave the best F1 for fewer parameters than LSTM, but took the longest to train in this run. More complexity did not guarantee better accuracy: LSTM had the most parameters and the lowest F1 within 8 epochs.

11. Effect of Sequence Length

Each model was retrained on the first T steps of every window (3 epochs per run).

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	76.91	20.49	72.62
64	77.78	53.46	55.56
128	62.85	61.11	62.51

Plot 6: Sequence Length vs. Test F1-score

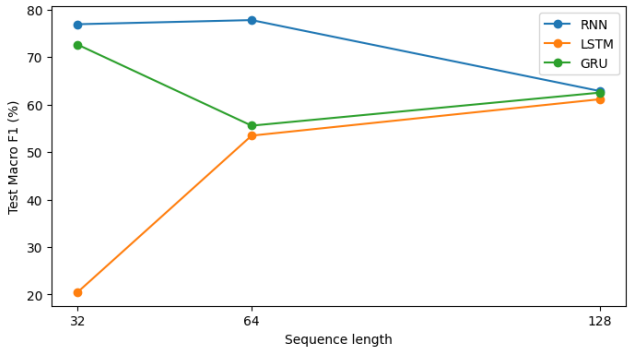


Figure 5: Test macro F1 against sequence length.

Inference: LSTM improved steadily with more context, while RNN and GRU did best on short windows. With only 3 epochs, longer sequences are harder to optimize, so more context helped only the model whose gates handle long inputs.

12. Video Understanding Using CNN and RNN



UCF101 and pretrained MobileNetV2 weights could not be downloaded in the execution environment, so a controlled three-class motion-video set was generated (Horizontal, Vertical, Diagonal; 60 videos per class). Ten 32×32 frames were sampled per video. A small CNN was first trained on a frame-position task, then frozen and used only as a feature extractor; only the GRU classifier was trained.

Item	Value
Videos (train / val / test)	126 / 27 / 27
Frames per video	10
CNN feature dimension D	256
Recurrent input tensor	(180, 10, 256), i.e. $X \in \mathbb{R}^{B\times 10\times 256}$

Plot 7: Video Sample Frames

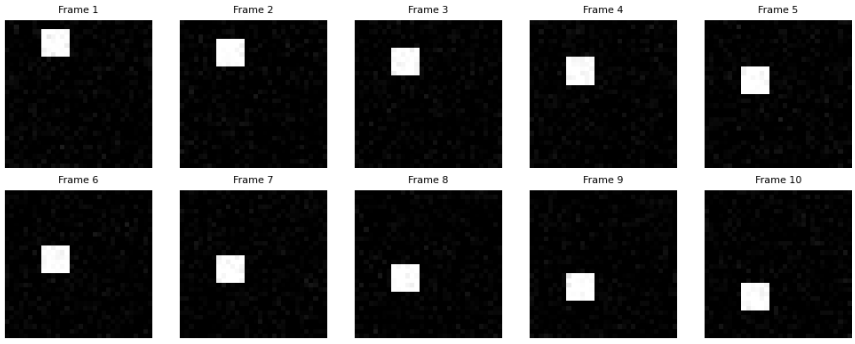


Figure 6: Ten sampled frames from one Vertical test video.

Inference: The CNN captures *where* the object is in each frame (spatial); the GRU learns *how* that position changes across frames (temporal), which is what separates the three motions.

Plot 8 and Plot 9: Video Training Curves and Confusion Matrix

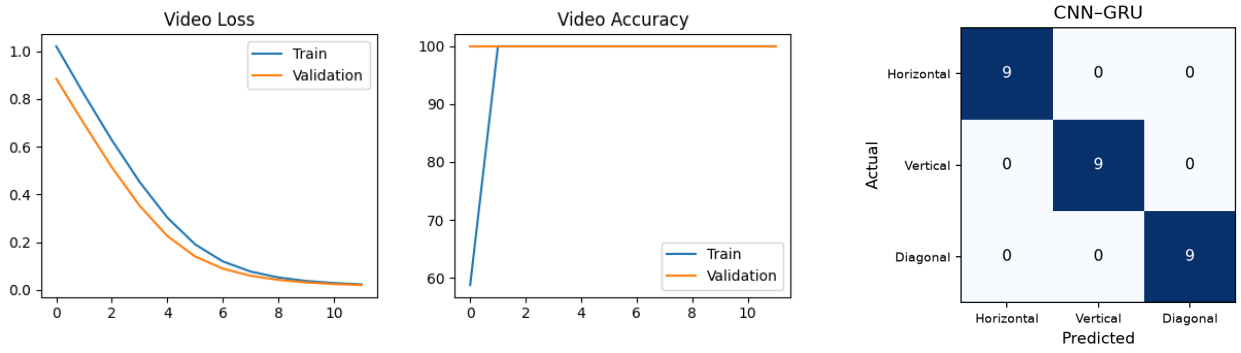


Figure 7: CNN–GRU loss and accuracy (left) and test confusion matrix (right).

Video Metric	Value
Accuracy	100.00%
Macro Precision	100.00%
Macro Recall	100.00%
Macro F1	100.00%
Trainable parameters	27,939
Training time	0.156 s

Model output (first test video)

```
Predicted class : Vertical
Actual class    : Vertical
Confidence      : 0.9819
```

Inference: Loss fell smoothly and validation accuracy was 100% from the first epoch. There were no temporal confusions because the three motion directions are cleanly separable; real UCF101 clips would be much harder.

13. Sequence-to-Sequence Learning



A synthetic reversal set of 1,800 sequences (length 4, integers 1–9) was split 1,260 / 270 / 270. The encoder and decoder were LSTMs with 32 units and a 16-dimensional embedding. The decoder used teacher forcing during training and its own predictions at test time.

Sample	Input	Predicted	Expected
1	[3, 2, 6, 5]	[5, 6, 2, 3]	[5, 6, 2, 3]
2	[8, 4, 8, 4]	[4, 8, 4, 8]	[4, 8, 4, 8]
3	[6, 8, 3, 3]	[3, 3, 8, 6]	[3, 3, 8, 6]
4	[6, 3, 6, 4]	[4, 6, 3, 6]	[4, 6, 3, 6]
5	[4, 4, 4, 1]	[1, 4, 4, 4]	[1, 4, 4, 4]

Seq2Seq Metric	Value
Token accuracy	98.80%
Sequence accuracy	97.78%
Final training loss	0.0859
Final validation loss	0.0897

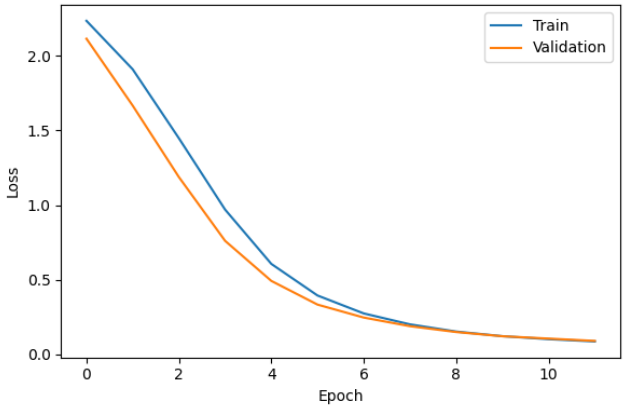


Figure 8: Encoder–decoder training and validation loss.

Inference: Both losses fell together, so the model converged without overfitting. Sequence accuracy is lower than token accuracy because one wrong token makes the whole sequence wrong.

14. Consolidated Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	86.11	92.42	86.11	83.19	2,006
LSTM	83.33	75.00	83.33	77.78	6,134
GRU	89.81	90.79	89.81	89.55	4,758
CNN–GRU (video)	100.00	100.00	100.00	100.00	27,939

Seq2Seq Model	Token Acc. (%)	Sequence Acc. (%)	Train Loss	Val Loss
Encoder–decoder LSTM	98.80	97.78	0.0859	0.0897

15. Discussion Questions

No.	Question	Answer
1	What is a recurrent neural network?	A network that processes a sequence step by step while carrying a hidden state.
2	What does the hidden state represent?	A summary of the inputs seen so far.
3	Why is sequence order important?	The meaning lies in how values change over time.
4	Feed-forward network vs. RNN?	A feed-forward net sees one fixed input; an RNN reuses weights across time and has memory.
5	Explain BPTT.	Unroll the RNN over time and backpropagate the loss through every step.
6	Why can gradients vanish?	Repeated multiplication by factors below 1 shrinks them towards zero.
7	Why can gradients explode?	Repeated multiplication by factors above 1 makes them grow very large.
8	What are long-term dependencies?	Links between an output and inputs many steps earlier.
9	Role of the LSTM cell state?	A memory path updated additively, so information survives many steps.
10	LSTM forget, input and output gates?	Forget removes memory, input adds new memory, output controls what is exposed as h_t.
11	GRU reset and update gates?	Reset controls how much history enters the candidate; update mixes old and new state.
12	Compare LSTM and GRU.	LSTM: 3 gates and a cell state. GRU: 2 gates and no cell state, so fewer parameters.
13	Why is GRU's gating simpler?	It merges the forget/input gates into one update gate and drops the cell state.
14	Why use the same settings for all three?	So differences come from the recurrent layer, not the setup.
15	What does the confusion matrix reveal?	Which classes are confused, e.g. LSTM never predicted WALKING_UPSTAIRS.
16	Why report macro F1?	It weights every class equally, so a failed class is not hidden.
17	Significance of sequence length?	It sets how much temporal context the model sees and how hard training is.
18	Why can a CNN extract video features?	It turns each frame into a compact spatial feature vector.
19	What does the CNN extract?	Spatial information: shapes, edges and object position in one frame.
20	What does the recurrent network learn?	Temporal information: how frame features change over time.
21	Why arrange CNN features as a sequence?	The recurrent layer needs ordered inputs to model motion.
22	Encoder and decoder in seq2seq?	The encoder compresses the input into a context state; the decoder generates the output from it.
23	What is teacher forcing?	Feeding the true previous token to the decoder during training.
24	Token vs. sequence accuracy?	Token: correct positions. Sequence: fully correct sequences only.
25	Why keep the test set untouched?	To get an unbiased estimate of generalization.

16. Conclusion

GRU gave the best HAR result (89.55% macro F1) with fewer parameters than LSTM, and WALKING vs. WALKING_UPSTAIRS was the main error for every model. The BPTT values matched the program, the CNN–GRU pipeline separated motion from frame features, and the encoder–decoder reversed sequences with 97.78% sequence accuracy. The HAR and video data were generated locally, so these results show the methods rather than benchmark performance.

17. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. D. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. K. Soomro, A. R. Zamir and M. Shah, "UCF101: A Dataset of 101 Human Action Classes from Videos in the Wild," 2012.
6. PyTorch Documentation: `https://pytorch.org/docs`

18. Source Code

GitHub repository	`https://github.com/Joshua-Divide/LAB`
Notebook	`Lab 6/Experiment 6.ipynb`